F1 TELEMETRY ANALYZER

MANUAL TÉCNICO DO SISTEMA

Documentação técnica conforme estrutura e apresentação ABNT

Sistema de análise de telemetria, Racing Line, comparação de voltas, modelo físico, coach de corrida e integração com Assetto Corsa

São Paulo
2026

F1 TELEMETRY ANALYZER

MANUAL TÉCNICO DO SISTEMA

Manual técnico elaborado para explicar, de forma clara e completa, a arquitetura, os fluxos de dados, as integrações, os módulos de análise e os procedimentos de uso do sistema F1 Telemetry Analyzer no estado atual do projeto.

Elaboração: documentação gerada a partir do repositório local do sistema.

São Paulo
2026

RESUMO

Este manual descreve o sistema F1 Telemetry Analyzer no estado atual do desenvolvimento. A aplicação combina backend FastAPI, frontend React, telemetria em tempo real, integração por memória compartilhada com Assetto Corsa, geometria de pista obtida de arquivos KN5/AI, Racing Line de referência baseada na melhor volta válida, comparação por microsetores, modelo físico simplificado de perfil de velocidade e painéis de apoio à pilotagem.

O foco deste documento é permitir que uma pessoa que não participou do desenvolvimento entenda o que o sistema faz, como os dados circulam, como executar a aplicação, quais módulos compõem a solução, quais riscos já foram tratados e quais limitações ainda existem.

Palavras-chave: telemetria; automobilismo; Assetto Corsa; Racing Line; FastAPI; React.

LISTA DE SIGLAS

Quadro 1 - Siglas e significados

Sigla	Significado no sistema
ABNT	Associação Brasileira de Normas Técnicas.
AC	Assetto Corsa, simulador de corrida usado como fonte real de telemetria.
API	Interface de programação usada pelo frontend para consultar o backend.
CSV	Arquivo tabular usado para telemetria gravada ou dados de pista legados.
KN5	Formato de malha 3D de pistas/carros do Assetto Corsa.
RL	Racing Line, linha de referência usada para comparação do piloto.
UI	Interface de usuário no navegador.
UDP	Protocolo usado para receber dados de oponentes e capturas de outros jogos.
WS	WebSocket usado para transmitir telemetria ao frontend.

Fonte: elaborado a partir do código-fonte do projeto.

SUMÁRIO

1 Introdução
2 Visão geral do sistema
3 Requisitos e tecnologias
4 Estrutura do projeto
5 Arquitetura do backend
6 Pipeline de telemetria
7 Racing Line e análise de microsetores
8 Physics Fast Lap
9 Inteligência de pilotagem e painéis de apoio
10 Arquitetura do frontend
11 Oponentes, gravação e persistência
12 Execução operacional
13 API do sistema
14 Testes, validação e baseline
15 Troubleshooting
16 Limitações, decisões técnicas e roadmap
Referências
Apêndice A - Variáveis de ambiente
Apêndice B - Glossário técnico

Fonte: organização do próprio manual.

1 INTRODUÇÃO

1.1 Finalidade

O F1 Telemetry Analyzer é uma estação de análise de pilotagem para simulação de corrida. Ele lê telemetria do carro, projeta a posição na geometria da pista, compara voltas, identifica perdas por trecho e apresenta diagnósticos visuais no mapa. A meta prática é dar ao piloto uma leitura rápida de onde freia, acelera, posiciona o carro e perde tempo.

A primeira Racing Line do sistema não é uma linha perfeita matemática. Ela é uma linha de referência real, retirada da melhor volta válida disponível. Essa decisão torna a ferramenta útil mais cedo: o piloto compara seu comportamento atual com uma volta real já realizada antes de evoluir para uma linha ideal composta.

1.2 Público-alvo

- Pilotos de simulador que querem entender sua volta com leitura visual e numérica.
- Desenvolvedores que precisam manter ou expandir o backend, o frontend e as integrações.
- Usuários técnicos que precisam operar o sistema com Assetto Corsa e dados gravados.

1.3 Escopo

Este manual cobre o estado atual do sistema até 2 de junho de 2026. Ele descreve módulos existentes, endpoints, fluxos de dados, formatos, modos de execução, testes e limitações. Não cobre instalação avançada de mods do Assetto Corsa nem substitui a documentação oficial das normas ABNT; apenas aplica as convenções de apresentação, estrutura e referência cabíveis.

2 VISÃO GERAL DO SISTEMA

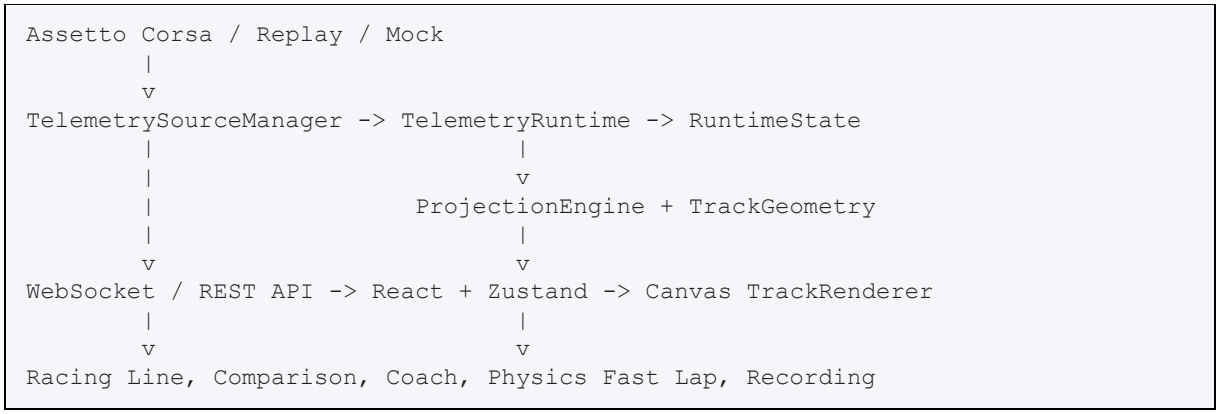
2.1 O que o sistema faz

- Recebe telemetria ao vivo do Assetto Corsa ou usa fontes mock/replay para desenvolvimento.
- Carrega ou reconstrói a geometria da pista e mantém a pista em um espaço canônico X/Z.
- Projeta o carro na pista e calcula distância percorrida, offset lateral e progresso da volta.
- Seleciona a melhor volta válida como Racing Line de referência.
- Compara volta atual, referência e oponentes em microsetores configuráveis.

- Exibe a Racing Line como linha contínua e, opcionalmente, como gradiente de velocidade/freio.
- Gera relatório de Physics Fast Lap sobre a Racing Line existente.
- Mostra painéis de física, comparação, coach, telemetria, oponentes e performance.

2.2 Arquitetura em camadas

Figura 1 - Fluxo lógico do sistema



Fonte: elaborado a partir do código-fonte do projeto.

2.3 Princípios de projeto

- A geometria da pista é tratada como referência estável. O carro e os overlays usam o mesmo espaço de mapa.
- O overlay da Racing Line não tem projeção própria; ele reutiliza a transformação mundo X/Z para mapa com Y = -Z.
- A linha visual principal usa amostras reais da volta de referência, sem suavização agressiva que corte curvas.
- Microsetores são usados para análise e tabela, não para desenhar a linha principal como pista segmentada.
- O backend inicia em modo seguro sem auto-conectar ao Assetto, evitando criar memória compartilhada antes do simulador.
- Os cálculos pesados e payloads grandes são desacoplados por cache, polling controlado e modos de performance.

3 REQUISITOS E TECNOLOGIAS

3.1 Requisitos de execução

Quadro 2 - Requisitos principais

Item	Descrição
Sistema operacional	Windows é o ambiente principal validado; Linux/macOS são suportados para fluxos sem Assetto Corsa.
Python	Python 3.9+; no workspace atual há ambiente virtual em .venv.

Node.js	Node.js 18+ e npm para o frontend React/Vite.
Assetto Corsa	Necessário para telemetria real por memória compartilhada.
Navegador	Frontend em http://localhost:5173.
Backend	FastAPI em http://localhost:8000.

Fonte: elaborado a partir do código-fonte do projeto.

3.2 Tecnologias do backend

Quadro 3 - Bibliotecas Python

Biblioteca	Uso
FastAPI e Uvicorn	API REST, WebSocket e servidor ASGI.
Pandas, NumPy e SciPy	Leitura de CSV, cálculo vetorial, estatística e interpolação.
Pydantic	Estruturas e validação de dados.
DuckDB e PyArrow	Persistência local e dados tabulares/colunares.
FastF1	Integração histórica com dados reais de F1.
ONNX Runtime, scikit-learn, PyTorch, Transformers	Base para módulos de IA, modelos e experimentos futuros.
Gymnasium e NetworkX	Ambiente de RL e grafos de conhecimento/memória.

Fonte: backend/requirements.txt.

3.3 Tecnologias do frontend

Quadro 4 - Bibliotecas JavaScript/TypeScript

Biblioteca	Uso
React 18	Composição da interface e painéis.
Vite	Servidor de desenvolvimento e build.
Zustand	Store global de telemetria e estado da UI.
Axios	Cliente HTTP para endpoints REST.
Lucide React	Ícones dos controles e painéis.
Plotly/Recharts	Gráficos e componentes históricos ainda disponíveis.
TypeScript	Tipos do store, painéis e payloads principais.

Fonte: frontend/package.json.

4 ESTRUTURA DO PROJETO

Quadro 5 - Diretórios e responsabilidades

Caminho	Responsabilidade
backend/main.py	Aplicação FastAPI, endpoints, ciclo de vida, runtime e orquestração geral.

backend/core	Módulos de telemetria, pista, física, análise, IA, gravação e integração com Assetto.
backend/tests	Testes unitários dos módulos críticos.
frontend/src	Aplicação React, store, componentes e cliente de API.
frontend/src/components/map	Renderização do mapa, carro, oponentes, pista e Racing Line em canvas.
data/cache/tracks	Geometrias de pista cacheadas, especialmente Interlagos KN5.
data/telemetry_db e data/feature_store	Bases DuckDB locais para telemetria e features.
tools/assetto_opponents_exporter	Aplicativo Python do Assetto para exportar oponentes via UDP.
outputs	Artefatos de validação, logs e capturas geradas durante desenvolvimento.

Fonte: elaborado a partir do código-fonte do projeto.

5 ARQUITETURA DO BACKEND

5.1 Aplicação FastAPI

O arquivo backend/main.py centraliza a aplicação. Ele cria objetos globais de runtime, cache, buffer de telemetria, estado da pista, runtime de oponentes, gravação e runtime cognitivo. Durante o lifespan, o sistema seleciona a fonte de telemetria, inicializa a geometria da pista, inicia a thread de leitura e registra eventos para WebSocket, gravação e painéis.

5.2 Fontes de telemetria

O TelemetrySourceManager escolhe entre mock, replay CSV e Assetto Corsa. O modo auto agora não tenta conectar ao Assetto por padrão. Essa escolha foi feita para impedir que a aplicação crie ou segure páginas de memória compartilhada antes do simulador estar pronto.

Quadro 6 - Fontes de telemetria

Fonte	Quando usar	Comportamento
mock	Desenvolvimento e inicialização segura.	Não lê AC e mantém a aplicação estável com geometria padrão.
replay	Reproduzir CSV de telemetria.	Carrega amostras de data/example_telemetry.csv ou outro fixture.
assetto_corsa	Sessão real aberta na pista.	Exige leitura válida antes de trocar a fonte; se não houver amostra, falha com mensagem clara.

auto	Inicialização geral.	Com ASSETTO_AUTO_CONNECT=0 seleciona mock; com 1 pode promover para Assetto quando houver amostra real.

Fonte: backend/core/telemetry/telemetry_reader_impl.py.

5.3 Integração segura com Assetto Corsa

A integração com Assetto usa as páginas de memória compartilhada acpmf_physics, acpmf_graphics e acpmf_static. O Assetto deve ser o dono dessas páginas. Por isso, o adaptador primeiro chama OpenFileMappingW para verificar se elas já existem; só depois abre mmap em modo leitura. Se alguma página não existir, a conexão retorna falso e não cria nomes de memória por acidente.

Fluxo operacional recomendado: abrir o Assetto Corsa, entrar em uma sessão de pilotagem, iniciar ou manter o backend em modo seguro, conectar manualmente a fonte assetto_corsa e, antes de fechar ou reiniciar o Assetto, voltar a fonte para mock para liberar handles.

5.4 Estado em tempo real e projeção

O RuntimeState guarda a pista ativa, o último sample, o estado projetado do carro e o motor de projeção. Cada TelemetrySample é convertido para uma estrutura de UI contendo worldPosition, mapPosition, projectedPosition, distanceAlongTrack, lateralOffset, velocidade, pedais, marcha, RPM e dados de física. Essa estrutura alimenta WebSocket e endpoints REST.

A convenção espacial é fundamental: a amostra canônica usa coordenadas de mundo X/Z. Para o mapa, o eixo visual Y é calculado como -Z. O carro, a pista, os oponentes e a Racing Line precisam usar essa mesma convenção para evitar espelhamento ou desalinhamento.

5.5 Geometria de pista

A geometria principal de Interlagos vem dos arquivos do Assetto Corsa. O provider KN5 Surface Track Geometry lê a malha da pista, cruza com fast_lane.ai e calcula bordas por intervalos de superfície. O cache atual de vhe_interlagos gp possui aproximadamente 4.345,88 m, loop fechado, 2.680 amostras válidas de raycast e largura média aproximada de 12,50 m.

Quando a fonte é mock, o backend carrega a geometria fixa de Interlagos para permitir uso e testes sem Assetto. Quando a fonte é Assetto, o sistema tenta

carregar KN5/cache; reconstrução por trajetória do piloto fica desabilitada para evitar que uma volta imperfeita vire geometria oficial.

5.6 Coleta de volta e buffer

O TelemetryBuffer mantém até 20.000 amostras. O LapCollector detecta o fechamento da volta pelo wrap de normalizedSplinePosition, de valores altos próximos a 0,95 para baixos próximos a 0,10. Uma volta candidata precisa ter amostras suficientes, distância plausível e área de pista mínima.

6 PIPELINE DE TELEMETRIA

6.1 Etapas do processamento

1. A fonte ativa produz um TelemetrySample: mock, replay ou Assetto Corsa.
2. O TelemetryRuntime lê amostras em thread própria, normalmente a 60 Hz.
3. A amostra entra no buffer e no LapCollector.
4. O RuntimeState projeta a posição na pista e cria o frame de UI.
5. O event_bus emite processed_frame para WebSocket, gravação e modelo cognitivo.
6. O frontend recebe o frame, normaliza o payload e atualiza o store em intervalos controlados.
7. O mapa desenha carro, pista, histórico, oponentes e overlays sem depender de re-render React por frame.

6.2 Modelo canônico de amostra

TelemetrySample é a unidade central do sistema. Ela guarda posição de mundo, velocidade em km/h, yaw, progresso normalizado, volta, setor, throttle, brake, steering, marcha, RPM, acelerações, dados de pneus, suspensão, combustível, DRS, ABS, TC, temperatura e superfície quando disponíveis. Campos ausentes são aceitos; os módulos classificam a completude dos dados em vez de quebrar.

7 RACING LINE E ANÁLISE DE MICROSETORES

7.1 Seleção da linha de referência

A Racing Line atual é baseada na melhor volta válida já completada pelo piloto. O sistema ignora a volta atual, rejeita voltas com poucas amostras, cobertura insuficiente do progresso, duração menor que 20 s ou maior que 900 s, e seleciona a menor duração válida como BEST_LAP.

7.2 Linha visual

A linha visual não é desenhada a partir dos 50 microsetores. Ela usa visualLine.points, isto é, amostras reais da volta de referência ordenadas por splinePosition. Isso evita aparência de pista segmentada e evita que uma suavização artificial corte curvas como o S do Senna.

7.3 Microsetores

Microsetores são divisões analíticas da volta. Por padrão, o usuário escolhe 20, 50 ou 100. Cada microsetor tem setor macro, início/fim de spline, velocidade média da referência, zona de freio, zona de aceleração, curvatura estimada, confiança e amostras usadas.

7.4 Comparação com o piloto

A comparação calcula velocidade média do piloto, velocidade da referência, delta de velocidade, desvio médio de trajetória, freio/aceleração do piloto e da referência, tempo estimado e issue principal. As classes mais importantes são trajetória, freio cedo, freio tarde, aceleração tardia, baixa velocidade de contorno e baixa velocidade de saída.

7.5 Overlay no mapa

Quadro 7 - Modos da Racing Line no canvas

Modo	Função
LINE	Mostra a linha fina, contínua e limpa da volta de referência.
DIAG	Adiciona marcadores de problemas em cima da linha principal.
PEDAL	Marca zonas de freio e aceleração da referência.
RAW	Mostra pontos reais da volta de referência.
PTS	Mostra pontos dos microsetores.
SEGS	Mostra conexões segmentadas para debug.
SMTH	Mostra linha suavizada apenas como camada de diagnóstico.
SPEED	Mostra degradê por velocidade/freio: vermelho para frenagem/baixa velocidade e verde para alta velocidade.

Fonte: frontend/src/components/map/RacingLineOverlay.jsx.

8 PHYSICS FAST LAP

8.1 Objetivo

Physics Fast Lap estima um perfil de velocidade alvo sobre a Racing Line existente. Ele não tenta criar uma linha ideal nova. A proposta é responder: dada esta linha real de referência, qual seria um perfil de velocidade fisicamente plausível para ela?

8.2 Parâmetros atuais

Quadro 8 - Parâmetros do modelo físico V1

Parâmetro	Valor atual	Origem
Máximo lateral	1,55 g	CONFIG_DEFAULT
Aceleração máxima	4,2 m/s²	CONFIG_DEFAULT
Frenagem máxima	10,0 m/s²	CONFIG_DEFAULT
Velocidade de reta	95 m/s	limite interno do modelo
Massa	Indisponível	UNAVAILABLE
Grip real da superfície	Indisponível quando não vem do AC	UNAVAILABLE
Temperaturas	Indisponíveis para cálculo de alvo	UNAVAILABLE

Fonte: backend/core/physics_lap_model.py.

8.3 Como o alvo é calculado

8. Extrai amostras da visualLine da Racing Line.
9. Calcula distância acumulada entre pontos reais.
10. Estima curvatura com três pontos e suavização leve.
11. Calcula limite de velocidade por curvatura usando v = sqrt(a_lateral / curvatura).
12. Aplica passagem para frente respeitando aceleração máxima.
13. Aplica passagem para trás respeitando frenagem máxima.
14. Agrega resultado por microsetores e compara piloto, referência e alvo físico.

O relatório marca avisos importantes: physics_model_v1_simplified, uses_existing_reference_racing_line e not_a_full_assetto_corsa_physics_simulation. Portanto, o resultado deve ser lido como um alvo operacional inicial, não como simulação completa de dinâmica veicular.

9 INTELIGÊNCIA DE PILOTAGEM E PAINÉIS DE APOIO

9.1 Race Coach

O Race Coach transforma segmentos da Racing Line em mensagens acionáveis. Ele agrupa microsetores consecutivos com o mesmo problema, calcula severidade e confiança, gera evidências e recomendações. Exemplos: atrasar levemente a frenagem, estabilizar antes do ápice, preparar a entrada para acelerar mais cedo ou comparar entrada/saída com a linha no mapa.

9.2 Comparação ao vivo

O módulo de comparação ao vivo usa a volta atual, uma referência válida e histórico de oponentes. Ele compara velocidade, delta, trajetória, freio e aceleração por microsetor. Para oponentes, quando canais de pedal não existem, usa tendência de velocidade como inferência.

9.3 Física do carro

O painel de física separa dados reais, inferidos e indisponíveis. Para o piloto, usa canais do AC quando disponíveis: velocidade, acelerações, pedais, pneus, suspensão, ambiente e estado do carro. Para oponentes, normalmente há velocidade e posição, com aceleração/frenagem inferidas por tendência.

9.4 Modelo cognitivo

O CognitiveRuntime é leve e roda com throttle de emissão. Ele mede confiança, agressividade, suavidade e consistência a partir de variações de throttle, freio e direção. O DriverStateEstimator classifica estados como steady, overdriving, hesitating, confident_push ou unstable.

10 ARQUITETURA DO FRONTEND

10.1 Layout

O Dashboard organiza a interface em mapa, painel direito e aba lateral esquerda de telemetria. A telemetria esquerda pode ser aberta ou ocultada. O painel direito possui abas Engineer, Debrief, Compare, Line e Physics. O mapa ocupa a região central e o controle de Replay fica no rodapé.

10.2 Store e WebSocket

O useTelemetryStore mantém latestFrame, histórico, volta atual, volta anterior, voltas completadas, oponentes, eventos de coach, falas do engenheiro, estado cognitivo e modo de performance. O hook useTelemetryWS conecta em ws://hostname:8000/ws, normaliza mensagens e usa buffers pendentes para reduzir re-renders em React.

10.3 Renderização do mapa

TrackRenderer usa canvas e requestAnimationFrame. O carro é suavizado com buffer temporal, atraso controlado de render, interpolação, extrapolação curta e reset em saltos grandes. O objetivo é reduzir travadas visuais sem falsificar a telemetria original.

10.4 Modos de performance

Quadro 9 - Modos de performance do frontend

Modo	Efeito
QUALITY	Atualiza store com maior frequência e mantém histórico visual maior.
BALANCED	Modo padrão; busca equilíbrio entre fluidez e custo de renderização.
PERFORMANCE	Aumenta intervalos, reduz histórico e força Racing Line simples para aliviar o canvas.

Fonte: frontend/src/store/useTelemetryStore.ts e frontend/src/components/map/TrackRenderer.jsx.

11 OPONENTES, GRAVAÇÃO E PERSISTÊNCIA

11.1 Oponentes

O sistema possui um aplicativo Python para Assetto Corsa chamado ac_opponents_exporter. Ele envia snapshots UDP para 127.0.0.1:8765 com carros detectados, excluindo o player. O backend recebe, valida, remove dados obsoletos após alguns segundos e expõe /api/live/opponents.

11.2 Gravação de sessão

RecordingRuntime assina eventos de telemetria do jogador, oponentes, splits, voltas e coaching. SessionRecorder grava JSONL em data/recordings, com metadata.json, player.jsonl, opponents.jsonl e events.jsonl. O gravador usa fila, batch, flush periódico e contabiliza frames descartados.

11.3 Persistência local

O projeto contém bancos DuckDB para telemetria e feature store, caches FastF1, caches de geometria de pista e arquivos de debug. Os caches de pista evitam recalcular geometria KN5 a cada inicialização.

12 EXECUÇÃO OPERACIONAL

12.1 Inicialização segura

```
cd C:\Users\Dokas\Desktop\f1-telemetry-analyzer\backend
$env:TELEMETRY_SOURCE='auto'
$env:ASSETTO_AUTO_CONNECT='0'
..\.venv\Scripts\python.exe main.py

cd C:\Users\Dokas\Desktop\f1-telemetry-analyzer\frontend
npm.cmd run dev
```

Fonte: procedimento validado durante o desenvolvimento.

12.2 Conectar ao Assetto

15. Abra o Assetto Corsa e entre em uma sessão de pilotagem.
16. Confirme que o carro está carregado na pista, não apenas no menu.
17. Com o backend rodando, envie POST /api/live/source/assetto_corsa.
18. Verifique GET /api/live/source e GET /api/live/telemetry.

```
Invoke-RestMethod -Method Post
http://localhost:8000/api/live/source/assetto_corsa
Invoke-RestMethod http://localhost:8000/api/live/source
```

12.3 Desconectar antes de fechar ou reiniciar o Assetto

```
Invoke-RestMethod -Method Post http://localhost:8000/api/live/source/mock
```

Esse passo libera os handles de memória compartilhada do backend. Ele é recomendado antes de fechar ou reiniciar o Assetto, especialmente após o problema de crash em SharedMemoryWriter identificado durante a fase de integração.

13 API DO SISTEMA

Quadro 10 - Endpoints principais

Endpoint	Método	Função
/health	GET	Status geral da aplicação.
/ws	WebSocket	Stream de telemetria e eventos para o frontend.
/api/live/source	GET	Mostra fonte ativa, reader, amostras e metadados.

/api/live/source/{source}	POST	Seleciona mock, replay ou assetto_corsa.
/api/live/telemetry	GET	Retorna frame atual e status de track/runtime.
/api/live/telemetry	POST	Ingere telemetria manual para testes.
/api/track/geometry	GET	Retorna geometria da pista para o mapa.
/api/track/cache	GET/POST	Consulta ou reconstrói cache de pista.
/api/live/comparison	GET	Comparação por microsetores.
/api/live/racing-line	GET	Payload da Racing Line e comparação.
/api/live/coach	GET	Relatório de coaching baseado na Racing Line.
/api/live/physics-fast-lap	GET	Perfil físico sobre a Racing Line existente.
/api/live/player-physics	GET	Física do piloto e oponentes.
/api/live/opponents	GET	Snapshot de oponentes recebidos via UDP.
/api/recording/status	GET	Status da gravação.
/api/recording/start	POST	Inicia gravação.
/api/recording/stop	POST	Para gravação.
/api/debug/performance	GET	Métricas de performance do backend.
/api/debug/projection	GET	Dados de debug da projeção espacial.
/api/debug/ac-shared-memory-full-inventory	GET	Inventário de campos AC.

Fonte: backend/main.py.

14 TESTES, VALIDAÇÃO E BASELINE

14.1 Testes de backend

Em 2 de junho de 2026, o backend foi validado com unittest discover no diretório backend/tests. Resultado: 72 testes executados em aproximadamente 1,24 s, todos aprovados.

```
Comando:
..\.venv\Scripts\python.exe -m unittest discover -s tests

Resultado resumido:
Ran 72 tests in 1.244s
OK
```

14.2 Build de frontend

O frontend foi validado com npm.cmd run build. O uso de npm.cmd evita bloqueio de política de execução do PowerShell sobre npm.ps1. Resultado: build

Vite concluído com sucesso, 1.500 módulos transformados, bundle JavaScript de 351,72 kB e 105,86 kB gzip.

```
Comando:
npm.cmd run build

Resultado resumido:
vite build
1500 modules transformed
built in 2.81s
```

14.3 Cobertura funcional testada

- AssettoAdapter não cria mmap nomeado quando a memória compartilhada ainda não existe.
- TelemetrySourceManager não auto-conecta ao Assetto por padrão e exige amostra válida na conexão manual.
- Racing Line seleciona a melhor volta válida, expõe visualLine real e rejeita voltas parciais.
- Physics Fast Lap calcula distância, curvatura, passes de aceleração/frenagem e cache de perfil.
- Race Coach, comparação, física, cognição, oponentes e gravação têm testes dedicados.

15 TROUBLESHOOTING

Quadro 11 - Problemas comuns

Sintoma	Causa provável	Ação recomendada
Assetto crasha em SharedMemoryWriter	Aplicação externa abriu/criou memória compartilhada antes do simulador.	Iniciar backend em modo seguro, abrir AC primeiro, conectar manualmente só em sessão e desconectar para mock antes de fechar AC.
Racing Line indisponível	Nenhuma volta anterior completa e válida.	Completar pelo menos uma volta limpa com cobertura suficiente.
Linha espelhada no mapa	Transformação mundo -> canvas diferente da do carro.	Garantir uso de X e Y=-Z, mesma função/convenção do mapPosition.
Linha parece pista segmentada	Uso visual dos microsetores como linha principal.	Usar visualLine.points como linha base e microsetores apenas como diagnósticos.
UI_RENDER_CRITICAL_FAILURE	Carga alta de canvas, payload grande ou re-render excessivo.	Usar modo PERFORMANCE, reduzir overlays, monitorar painel PERF e payloads.
Frontend não builda no PowerShell	ExecutionPolicy bloqueia npm.ps1.	Usar npm.cmd run build.
Sem oponentes	Exporter do Assetto não	Instalar app, habilitar no Content

	instalado/aberto ou sessão sem carros válidos.	Manager e verificar /api/live/opponents.
Physics Fast Lap sem dados	Racing Line ainda não está READY.	Gerar uma volta válida de referência primeiro.

Fonte: elaborado a partir do código-fonte do projeto.

16 LIMITAÇÕES, DECISÕES TÉCNICAS E ROADMAP

16.1 Limitações conhecidas

- A Racing Line atual é a melhor volta válida, não uma linha ideal matemática.
- Physics Fast Lap V1 é simplificado e não usa massa, aerodinâmica completa, pneus reais ou setup detalhado.
- Oponentes normalmente não expõem pedais; freio/aceleração são inferidos por tendência de velocidade.
- A geometria KN5 atual foi focada em Interlagos vhe_interlagos gp.
- A interface ainda depende de polling para alguns relatórios analíticos, como Racing Line e Physics Fast Lap.

16.2 Decisões recentes importantes

- Backend não auto-conecta ao Assetto por padrão: ASSETTO_AUTO_CONNECT=0.
- Shared memory do AC só é aberta se todas as páginas já existem.
- Linha visual da Racing Line usa amostras reais, sem suavização que corte curvas.
- Gradiente de Racing Line é uma opção de visualização, não substitui diagnósticos por microsetor.
- Left telemetry virou aba para abrir/ocultar e liberar espaço para leitura do mapa e painel direito.

16.3 Evolução recomendada

- Criar botão de UI para conectar/desconectar Assetto sem precisar de chamada manual de API.
- Evoluir Racing Line ideal composta a partir dos melhores microsetores de várias voltas.
- Adicionar suavização com restrição de curvatura mínima, preservando amostras reais onde necessário.
- Medir baseline antes/depois para qualquer otimização de canvas, payload ou re-render React.
- Adicionar exportação de relatórios PDF/HTML por sessão.
- Expandir suporte a pistas e validar o pipeline KN5 fora de Interlagos.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 14724: informação e documentação - trabalhos acadêmicos - apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 6023: informação e documentação - referências - elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 6024: informação e documentação - numeração progressiva das seções de um documento - apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 6027: informação e documentação - sumário - apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 6028: informação e documentação - resumo, resenha e recensão - apresentação. Rio de Janeiro: ABNT, 2021.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 10520: informação e documentação - citações em documentos - apresentação. Rio de Janeiro: ABNT, 2023.

F1 TELEMETRY ANALYZER. README.md, ARCHITECTURE.md, QUICKSTART.md, UDP_CAPTURE.md e CHANGELOG.md. Repositório local do projeto, 2026.

F1 TELEMETRY ANALYZER. Código-fonte do backend e frontend. Repositório local do projeto, 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT Catálogo. Disponível em: https://abntcatalogo.com.br/. Acesso em: 2 jun. 2026.

APÊNDICE A - VARIÁVEIS DE AMBIENTE

Quadro 12 - Variáveis de ambiente operacionais

Variável	Valor padrão/uso	Descrição
TELEMETRY_SOURCE	auto	Fonte inicial: auto, mock, replay ou assetto_corsa.
ASSETTO_AUTO_CONNECT	0/false	Quando falso, impede probe automático do Assetto.
ALLOW_REPLAY_FALLBACK	false	Permite replay quando auto não encontra fonte real.
TELEMETRY_DEBUG_REPLAY	false	Habilita replay fixture para debug.
TELEMETRY_RECORDING_ENABLED	true	Liga/desliga gravação local.
TELEMETRY_RECORDING_AUTO_START	true	Inicia gravação automaticamente no lifespan.
TELEMETRY_RECORDING_PLAYER_HZ	20	Frequência de gravação do player.
TELEMETRY_RECORDING_OPPONENTS_HZ	20	Frequência de gravação dos oponentes.
TELEMETRY_RECORDING_DIR	data/recordings	Diretório de saída das sessões.
VITE_API_URL	http://localhost:8000	Base URL usada pelo cliente Axios do frontend.

Fonte: elaborado a partir do código-fonte do projeto.

APÊNDICE B - GLOSSÁRIO TÉCNICO

Quadro 13 - Termos essenciais

Termo	Explicação simples
Racing Line	Linha que o piloto deve comparar visualmente. No estado atual, vem da melhor volta válida.
Microsetor	Pequeno trecho da volta usado para análise estatística e diagnóstico.
Spline position	Progresso normalizado da volta, de 0 a 1.
MapPosition	Posição convertida para o canvas do mapa.
ProjectionEngine	Módulo que encontra o ponto mais próximo do carro na pista.
KN5	Arquivo de malha 3D usado pelo Assetto Corsa para pista/carros.
Shared memory	Memória que o Assetto Corsa publica para outros programas lerem telemetria.
LapCollector	Componente que identifica volta candidata e fechamento de volta.

Physics Fast Lap	Perfil de velocidade alvo calculado sobre a Racing Line existente.
Race Coach	Módulo que transforma perdas por microsetor em recomendações de pilotagem.

Fonte: elaborado a partir do código-fonte do projeto.